

LAPORAN PRAKTIKUM 4 - BRUTE FORCE DAN DIVIDE CONQUER

Mata Kuliah : Algoritma dan Struktur Data

Nama : Ahmad Rizal Ali Pasha

Kelas/Absen : TI 1-C / 02

Nim : 254107020146

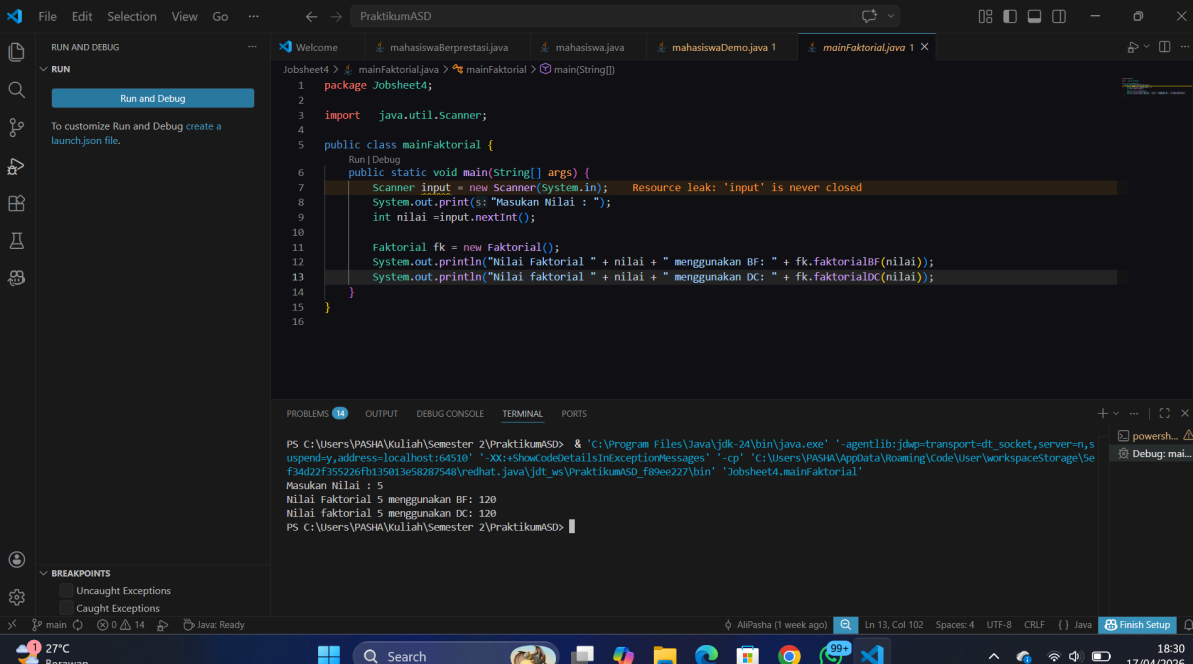
4.1 Tujuan Praktikum

Setelah melakukan materi praktikum ini, mahasiswa mampu:

1. Mahasiswa mampu membuat algoritma brute force dan divide-conquer
2. Mahasiswa mampu menerapkan penggunaan algoritma brute force dan divide-conquer

4.2 Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer

4.2.2. Verifikasi Hasil Percobaan



The screenshot shows an IDE with a Java file named `mainFaktorial.java`. The code defines a `mainFaktorial` class with a `main` method. It uses a `Scanner` to take input from the user. The input is then used to calculate the factorial using both Brute Force (BF) and Divide and Conquer (DC) methods. The output shows the factorial of 5 using both methods, resulting in 120.

```
1 package Jobsheet4;
2
3 import java.util.Scanner;
4
5 public class mainFaktorial {
6     public static void main(String[] args) {
7         Scanner input = new Scanner(System.in);
8         System.out.print("Masukan Nilai : ");
9         int nilai = input.nextInt();
10
11         Faktorial fk = new Faktorial();
12         System.out.println("Nilai faktorial " + nilai + " menggunakan BF: " + fk.faktorialBF(nilai));
13         System.out.println("Nilai faktorial " + nilai + " menggunakan DC: " + fk.faktorialDC(nilai));
14     }
15 }
16
```

The terminal output shows the execution of the program:

```
PS C:\Users\PASHA\Kuliah\Semester 2\PraktikumASD> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:64510' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\PASHA\AppData\Local\Temp\workspaceStorage\SeF44d2f35522a6fb135913e58287548\redhat_java\jdk_ws\PraktikumASD_f8ee227\bin' 'Jobsheet4.mainFaktorial'
Masukan Nilai : 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
PS C:\Users\PASHA\Kuliah\Semester 2\PraktikumASD>
```

4.2.3. Pertanyaan

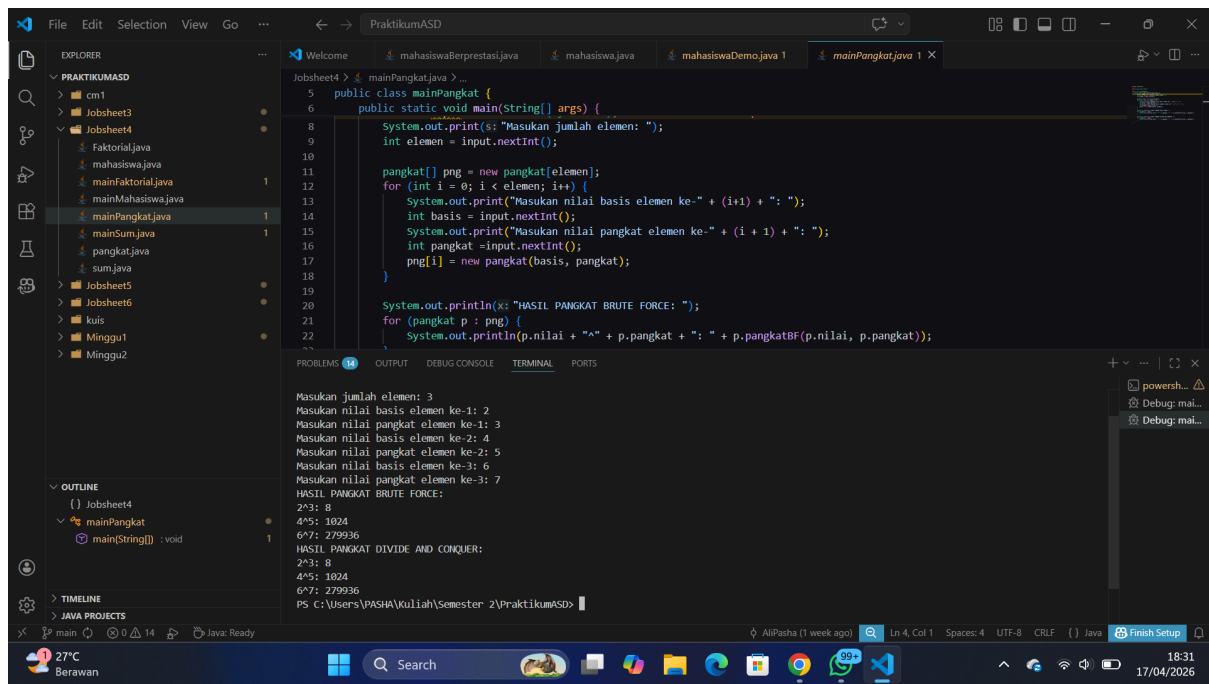
1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!

- if berperan sebagai base case jika kondisi tersebut belum terpenuhi maka dia akan lanjut ke else dan melakukan pemanggilan ulang fungsi nya sendiri sampai nilai base case yang berada di if terpenuhi
2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!
- sangat memungkinkan karena looping sendiri ada banyak jenis seperti while dan do while
 - bukti:

```
int faktorialBF(int n) {
    int fakto = 1, i = 1;
    while (i <= n) {
        fakto *= i;
        i++;
    }
    return fakto;
}
```
3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !
- fakto *=i menggunakan pendekatan burte force dengan melakukan looping, berbeda dengan int fakto = n * faktorialDC(n-1); yang menggunakan pendekatan rekursif yang mana dia akan terus memanggil fungsi nya sendiri sampai kondisi base case nya terpenuhi
4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!
- metode faktorial menggunakan bruteforce secara iteratif dengan perhitungan dilakukan secara langsung menggunakan looping dengan mengalikan bilangan yang ada secara berurutan. berbeda dengan faktorialdc menggunakan divide and conquer secara rekursif yang mana ia akan memecah masalah nya menjadi sub masalah yang lebih kecil sampai kondisi base case terpenuhi baru melakukan perhitungan secara berkala.

4.3 Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer

4.3.2. Verifikasi Hasil Percobaan

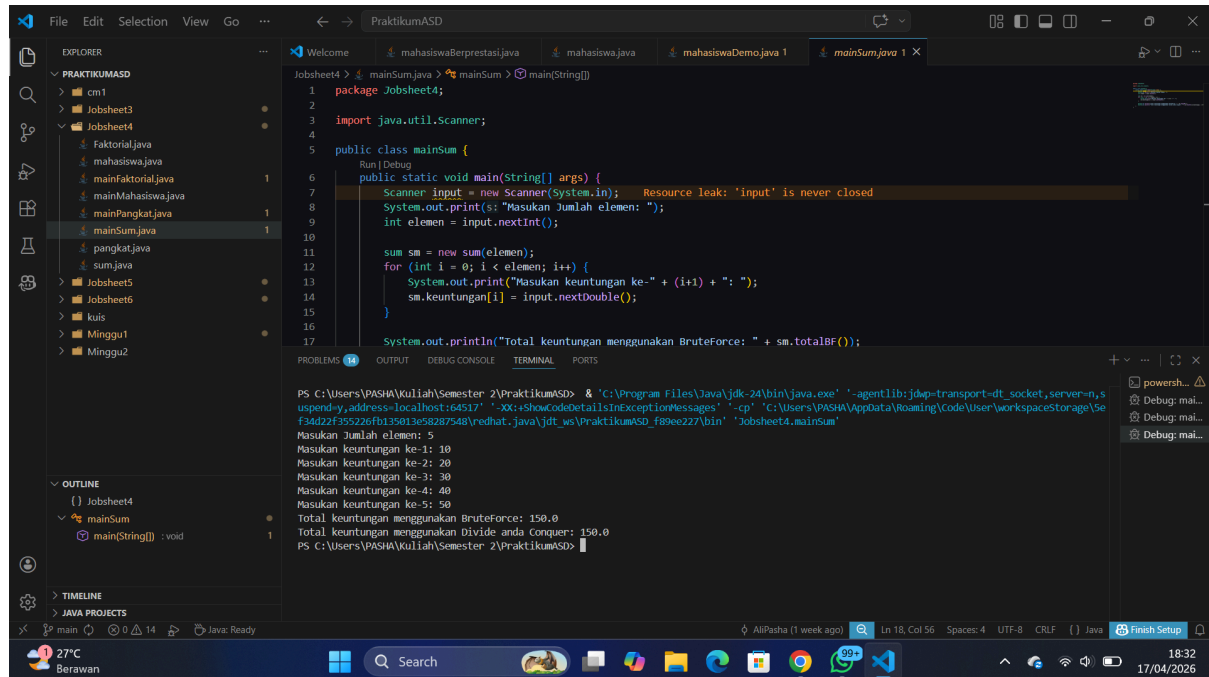


4.3.3. Pertanyaan

- Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!
 - pangkatBF() menggunakan bruteforce dengan loop linear untuk mengalikan basis sebanyak n kali secara berurutan
 - pangkatDC() menggunakan algoritma devide and conquer dengan algoritma membagi masalah jadi 2 bagian sehingga jumlah operasi perkalian jauh lebih efisien
- Apakah tahap combine sudah termasuk dalam kode tersebut? Tunjukkan!
 - Sudah. Tahap combine terjadi pada operator perkalian * saat menggabungkan hasil pemecahan sisi kiri dan sisi kanan.
- Pada method pangkatBF() terdapat parameter untuk melewati nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?
 - Secara teknis kurang efisien karena data sudah tersedia di atribut. Method tersebut bisa dibuat tanpa parameter jika hanya ingin memangkatkan data yang ada di objek tersebut.
- Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!
 - Method pangkatBF() bekerja secara iteratif (Brute Force) dengan mengalikan basis satu per satu sebanyak \$n\$ kali dalam satu jalur linear. Sebaliknya, pangkatDC() bekerja secara rekursif (Divide and Conquer) dengan membagi pangkat menjadi dua bagian kecil ($n/2$), menyelesaikannya secara terpisah, lalu menggabungkan hasilnya kembali sehingga proses jauh lebih cepat dan efisien.

4.4 Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer

4.4.3. Verifikasi Hasil Percobaan



```
package Jobsheet4;

import java.util.Scanner;

public class mainSum {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukan Jumlah elemen: ");
        int elemen = input.nextInt();

        sum sm = new sum(elemen);
        for (int i = 0; i < elemen; i++) {
            System.out.print("Masukan keuntungan ke- " + (i+1) + ": ");
            sm.keuntungan[i] = input.nextDouble();
        }

        System.out.println("Total keuntungan menggunakan BruteForce: " + sm.totalBF());
    }
}
```

```
PS C:\Users\PASHA\Kuliah\Semester 2\PraktikumASD> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:64517' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\PASHA\AppData\Roaming\Code\User\workspaceStorage\5ef34d22f355226fb135813e58287548\redhat.java\jdt_ws\PraktikumASD_f80ee227\bin' 'Jobsheet4.mainSum'
Masukan Jumlah elemen: 5
Masukan keuntungan ke-1: 10
Masukan keuntungan ke-2: 20
Masukan keuntungan ke-3: 30
Masukan keuntungan ke-4: 40
Masukan keuntungan ke-5: 50
Total keuntungan menggunakan BruteForce: 150.0
Total keuntungan menggunakan Divide and Conquer: 150.0
PS C:\Users\PASHA\Kuliah\Semester 2\PraktikumASD>
```

4.4.4. Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?
 - Variabel mid dibutuhkan untuk menentukan titik tengah sebagai pembatas saat membagi array menjadi dua bagian (kiri dan kanan) dalam tahap Divide.
2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```
double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);
```

- Digunakan untuk menampung hasil penjumlahan dari sub-masalah sisi kiri (lsum) dan sub-masalah sisi kanan (rsum) yang dikerjakan secara rekursif dalam tahap Conquer.

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```

- Ini adalah tahap Combine, di mana hasil dari dua sub-bagian array digabungkan kembali untuk mendapatkan hasil akhir penjumlahan seluruh elemen.

4. Apakah base case dari totalDC()?

- base case nya adalah ketika nilai l itu sama dengan r yang berarti data yang dicek tinggal 1 elemen saja.

5. Tarik Kesimpulan tentang cara kerja totalDC()

- totalDC() bekerja dengan prinsip Divide and Conquer secara rekursif, di mana array dibagi menjadi dua bagian melalui titik tengah (Divide), lalu setiap bagian dijumlahkan secara terpisah (Conquer), hingga akhirnya seluruh hasil penjumlahan bagian tersebut digabungkan kembali (Combine) untuk mendapatkan total akhir